

Multi-source Spatiotemporal Data Fusion for Urban Human-Flow Anomaly Detection and Prediction System

Project Technical Report

Author: ZENG Shengjie

Completion Date: 2026-08-17

Revision Date: 2026-09-05 (architecture description & experimental figures revised based on source code)

Repository: [ZENShengjie/spatio-temporal-data-pipeline](#)

Runtime Environment: EC2 g4dn.xlarge (T4 GPU) / Ubuntu 22.04 / Python 3.12 / PyTorch 2.4.1+cu124

Abstract

Targeting urban-scale human-flow anomaly detection, this report presents a complete multi-source spatiotemporal data fusion system. The full pipeline covers data acquisition & cleaning, spatiotemporal feature engineering, deep-learning prediction modelling, four-paradigm anomaly detection, interpretability analysis, and API-oriented service deployment. Taking Beijing taxi GPS records as the core dataset, the system integrates multi-dimensional auxiliary information including meteorology, POIs and OSM road networks, forming a reusable end-to-end workflow from raw inputs to anomaly alerts. Seven baseline time-series models and three spatiotemporal joint architectures are systematically evaluated. A four-branch fusion anomaly-detection framework is constructed, consisting of statistical thresholding, prediction-error-based scoring, Variational Autoencoder (VAE) reconstruction and Transformer autoencoder. On the V3 injected test set (4% injection rate, mixed burst/drop/sustained patterns), the proposed system achieves a statistical-only $F1 = 0.791$ that lifts to $F1 = 0.9515$ via three-way fusion (statistical + STF true inference + VAE), i.e. +20% absolute over the best single detector. Cached API end-to-end latency is below 10 ms, demonstrating practical application value for smart-city scenarios.

Chapter 1 Project Background

1.1 Research Background and Significance

Urban human-flow anomaly detection constitutes a core technical component for smart-city construction. Urban transportation systems generate massive volumes of

GPS, sensor and mobile-payment records that encode human mobility patterns. When anomalous events occur (large-scale gatherings, natural disasters, traffic collisions, public-health emergencies), human-flow distributions across city grids deviate markedly from regular patterns. Timely detection and early warning of such anomalies are critical for traffic scheduling, public-safety assurance and emergency response.

Traditional anomaly detection largely relies on hand-crafted rules and simple statistics such as fixed-threshold alarming and year-on-year / month-over-month ratio monitoring. These approaches struggle with complex spatiotemporal dependencies and fail to fully capture periodic human-flow cycles and spatial propagation effects. Recent advances in deep learning for time-series forecasting and graph-structured data provide new technical pathways for urban-scale spatiotemporal anomaly detection.

This work originates from an urban-computing research project based on the Beijing taxi GPS dataset. The research objective is to implement an end-to-end system capable of automatic anomaly discovery, early warning and visualisation using multi-source spatiotemporal fusion.

1.2 Related Work

Spatiotemporal anomaly detection remains an active research topic within spatiotemporal data mining. Existing mainstream approaches can be grouped into four categories.

Statistical-based methods: Classic pipelines include ARIMA seasonal modelling and Facebook Prophet decomposition. These approaches identify anomalies by fitting statistical distributions to time-series observations. Strengths include high interpretability and computational efficiency; limitations stem from weak modelling capacity for complex non-linear spatial dependencies.

Graph-neural-network-based methods: GCN, GAT and STGCN utilise road-network or similarity graphs to capture spatial correlations. Such architectures yield strong performance for urban traffic forecasting but suffer from large parameter footprints and substantial training overhead.

Transformer-based methods: After major breakthroughs in natural-language processing, Transformer variants such as SpacetimeFormer have been adapted for time-series and graph-modelling tasks. Spatiotemporal decomposition attention together with environment tokens delivers competitive performance for city-wide forecasting.

Reconstruction-based methods (VAE, AE, GAN): These techniques assume normal observations lie on a low-dimensional manifold; anomalous samples yield large reconstruction loss. Widely deployed for video anomaly detection and industrial fault diagnosis, they remain comparatively under-explored for urban-scale spatiotemporal datasets.

1.3 Project Objectives

1. Establish a complete multi-source data acquisition-and-cleaning pipeline, processing public datasets including NASA remote-sensing products, OpenWeather meteorology, GeoNames POIs, NYC Taxi records and OSM road networks.
2. Implement multiple spatiotemporal forecasting models targeting Beijing taxi-flow prediction, supporting 48-hour rolling grid-level forecasts over a 32×32 spatial partition.
3. Build a multi-paradigm fusion anomaly-detection framework combining statistical thresholds, prediction residuals and deep reconstruction. Optimise fusion weights via grid-search on the validation set.
4. Enable model interpretability analysis with SHAP values and attention heatmaps to improve system transparency and trustworthiness.
5. Deliver service-oriented deployment via FastAPI interfaces and Streamlit interactive visualisation for real-world practical usage.

Chapter 2 Requirement Analysis

2.1 Business Scenarios

The target application is an anomaly early-warning platform for municipal traffic-management authorities. Representative use-cases are listed below.

Large-scale public events (concerts, sports matches, exhibitions) cause short-term surges of visitor density in specific districts. Without effective warning mechanisms, stampedes and severe congestion may arise. The system monitors taxi-derived human-flow and generates alerts upon detecting anomalous crowd accumulation.

Natural hazards including heavy rain, heatwaves and cold snaps reshape urban mobility patterns. Detected deviations support traffic-control departments to adjust public-transport operation strategies.

Regular rush-hour congestion occurs daily. The system distinguishes routine congestion from abnormal jams triggered by accidents by learning historical mobility baselines.

2.2 Functional Requirements

Data management: Support unified ingestion, cleaning and feature engineering for heterogeneous multi-source data (taxi GPS trajectories, meteorology, POI categories, road-network topology). Provide standardised interfaces for querying data by temporal and spatial ranges.

Forecasting and analysis: Provide multi-step spatiotemporal forecasting for grid-wise human-flow over the next 48 h. The model shall account for holidays and weather covariates and quantify prediction uncertainty.

Anomaly detection: Realise multi-paradigm anomaly detection including statistical rules, prediction-residual scoring and deep-reconstruction-based detection. Fusion weights shall be determined in a data-driven fashion instead of manual tuning.

Visualisation: Deliver intuitive visual modules: spatiotemporal heatmaps, anomaly timelines and model-attention distribution plots.

2.3 Non-Functional Requirements

Performance: End-to-end anomaly-detection latency ≤ 100 ms for real-time warning scenarios. Batch processing shall compute anomaly scores for 600 time-steps within 2 seconds.

Maintainability: Modular, well-commented source code. Core algorithms decoupled from data pipelines and API layers to facilitate independent unit testing and reuse.

Extensibility: Architecture supports plug-and-play integration of new anomaly-detection algorithms and new data sources. Flexible configuration for multi-way fusion combinations.

Chapter 3 Technical Approach

3.1 Overall Architecture

The system follows the design philosophy: **data-driven + model fusion + engineering-oriented deployment**. Five hierarchical layers are defined.

1. **Data Layer:** Multi-source data ingestion, cleaning and feature preprocessing: missing-value imputation, outlier filtering, temporal alignment and spatial gridding. Inputs: taxi-GPS trajectories, meteorology, POIs, OSM road networks.
2. **Feature Layer:** Generate spatiotemporal feature vectors.
 - Spatial: grid coordinates, adjacency, POI density, road-network accessibility
 - Temporal: hour-of-day, day-of-week, holiday flags, historical-period averages
 - Exogenous covariates: temperature, precipitation, wind speed, weather conditions
3. **Model Layer:** Two sub-modules: time-series forecasting models and anomaly-detection models.
 - Forecasting: ARIMA, Prophet, LSTM, GRU, GCN, GAT, STGCN, AGFormer, STF (11 registered model names including two ablation variants: `agformer_static`, `stf_loc_only`)
 - Anomaly detection: statistical thresholding, prediction-error scoring, VAE reconstruction, Transformer autoencoder
4. **Fusion Layer:** Weighted combination of outputs from multiple anomaly detectors.

Grid-search on the validation set maximises F1-score. Supports two-way / three-way / four-way configurable fusion.

5. **Service Layer:** Wrap trained models into deployable services. FastAPI exposes RESTful endpoints for anomaly detection, forecasting and health checks. Streamlit delivers interactive visualisation for real-time monitoring and historical-event playback.

3.2 Technology Stack

- Deep-learning framework: PyTorch 2.x for flexible dynamic computation graphs. PyTorch Geometric for graph-convolution implementations.
- Hyper-parameter optimisation: Optuna with TPE sampler and MedianPruner early-stopping strategy.
- Web API: FastAPI for asynchronous services and auto-generated API documentation. orjson for high-speed serialisation (3-5 × faster than standard Python json library). LRU cache for repeated-request acceleration.
- Model inference: Support both GPU-accelerated and CPU-only paths. `torch.inference_mode()` disables gradient computation for speed-up and memory reduction. Quantisation and operator fusion optimise CPU-mode throughput.

3.3 Iterative Development Timeline

- **Week 1:** Multi-source data acquisition; establish five data pipelines (NASA remote sensing, OpenWeather, GeoNames POI, NYC Taxi, OSM).
- **Week 2:** Data cleaning and feature engineering; aggregate raw taxi GPS into 32×32 grid-flow representations; construct unified temporal-spatial-exogenous feature tensors.
- **Week 3:** Baseline time-series experiments. Evaluate ARIMA, Prophet, LSTM, GRU, GCN, GAT, GRU+STGCN residual. Select GRU as the optimal baseline.
- **Week 4:** Spatiotemporal joint-modelling experiments for STGCN, AGFormer and STF. Ablation studies quantify component contributions. The lightweight STF yields the best overall performance with 222 K parameters.
- **Week 5: Implement four-paradigm anomaly-detection framework. Grid-search fusion hyper-parameters on validation data. On the V3 injected test set (4% injection rate), the three-way fusion V3 (statistical + STF true inference + VAE) reaches $F1 = 0.9515$.**
- **Week 6:** Service-oriented deployment: FastAPI backend + Streamlit frontend. Optimise with orjson serialisation and LRU cache; cached latency drops below 10 ms from original ~1000 ms.
- **Week 7:** Model optimisation and interpretability. Optuna hyper-parameter search reduces STF test-set MAE from 597.70 down to 297.55 (-50 %). SHAP and attention visualisation reveal model decision mechanisms.
- **Week 8:** Documentation audit and polishing. Fix placeholder content, update

README and introduce CI quality-gate workflows.

Chapter 4 Data Processing

4.1 Data Sources

Taxi-flow dataset: TaxiBJ published by Microsoft Research Asia. It covers Beijing taxi GPS records spanning July 2013 to April 2016, pre-aggregated at 30-minute temporal resolution onto a 32×32 spatial grid. This work uses segment P4: **2015-11-01 to 2016-04-10**, totalling 3888 time-steps.

Meteorology: OpenWeather API outputs including temperature, wind speed, atmospheric pressure, humidity and weather labels. Resampled and aligned to hourly timestamps; 30-min taxi-flow values are averaged for temporal matching.

POI data: GeoNames open-access database with ten POI categories (catering, shopping, education, medical services etc.). Grid-wise POI-density statistics serve as proxies for urban-functional zoning.

Road-network data: OpenStreetMap (OSM) extracts road topology, building footprints and traffic-facility locations to build adjacency graphs for graph-network inputs.

4.2 Data Cleaning

Missing-value handling: Theoretical total time-steps for segment P4: 3888; raw completeness 92.8 %. Missing timestamps concentrate around Chinese-New-Year holidays when taxi activity declines sharply. Imputation uses same-weekday-same-hour historical averages computed per grid to avoid cross-grid contamination.

Outlier removal: GPS positioning drift and implausible instantaneous velocity (> 120 km/h) are filtered: trajectories with displacement exceeding 50 km/h are discarded. Aggregated grid-flow is clipped by the $3\text{-}\sigma$ rule.

Temporal alignment: Taxi-flow: 30-min granularity; meteorology: hourly; holiday flags: daily. Taxi-flow is averaged to hourly resolution; holiday labels are expanded to match each time-step.

4.3 Spatial Gridding

The Beijing urban study area is partitioned into uniform 32×32 rectangular cells (~ 1.18 km $\times$ 1.33 km per grid), covering the area within the Fifth Ring Road. Grid indices follow row-major ordering from 0 to 1023, each mapped uniquely to (row, col).

Two adjacency definitions are provided:

1. Distance-based: Manhattan distance ≤ 1 defines direct neighbours (maximum 8 neighbours per grid).
2. Road-network-based: actual reachability derived from OSM topology.

4.4 Dataset Partitioning

Segment P4 (3888 h) is split as follows:

- **Training set:** $t \in [0, 2784)$: 2784 h for parameter fitting.
- **Validation set:** $t \in [2784, 3288)$: 504 h for hyper-parameter tuning, threshold search and fusion-weight optimisation. Contains Spring-Festival samples for evaluating generalisation under special-period conditions.
- **Test set:** $t \in [3288, 3888)$: 600 h reserved exclusively for final performance evaluation, strictly unseen during training and tuning.

Synthetic anomalies are injected into validation and test partitions at a target anomaly ratio of 4 %. Three anomaly patterns are implemented:

- **Surge:** flow amplified 50 %-200 %
- **Drop:** flow reduced 30 %-70 %
- **Sustained:** continuous deviation for 2-4 hours

Anomalies are randomly assigned to high-traffic core grids to simulate real-world spatial characteristics.

Chapter 5 Model Design

5.1 Baseline Time-Series Models

ARIMA: Order $(p=5, d=1, q=2)$ selected by AIC criterion. Strength: full interpretability; weakness: error accumulation over multi-step forecasting.

Prophet: Trend-period-holiday decomposition. Excellent holiday-effect modelling; limited by one-shot prediction without iterative self-correction for rolling-forecast workflows.

LSTM: Two-layer architecture with 256 hidden units per layer, capturing long-range temporal dependencies via forget / input / output gates.

GRU: Simplified gated recurrent unit with fewer parameters than LSTM while maintaining comparable performance. Training time only 54 seconds; $t+1$ prediction accuracy reaches 94.52 %.

GCN: Spectral-domain graph convolution over spatial-distance-defined graph nodes; receptive-field constrained by static graph topology.

GAT: Graph-attention network, adaptively learns neighbour importance weights; superior performance for spatially-heterogeneous districts compared to vanilla GCN.

GRU+STGCN residual stack: GRU extracts temporal patterns; subsequent STGCN captures spatiotemporal joint signals; residual connections mitigate deep-network gradient vanishing.

5.2 Spatiotemporal Joint Modelling Architectures

Model	Input Dim	Core Operators	Param Count	Key Design Features
STGCN	F=7, T=48	GCNConv + causal GLU-TCN	~200 K	Static graph + 4 ST-Conv blocks
AGFormer	F=7, T=48	Top-K(K=16) spatial attention + temporal attention	~2.26 M	Adaptive graph initialised from static topology
STF	F=7, T=48	Env-token + Loc-token + Cross-Attention	222 K	Spatiotemporal decomposition + Conv1D

5.2.1 STGCN (Spatial-Temporal Graph Convolutional Network)

Sandwich-block paradigm: each `STGCNBlock` executes **spatial graph convolution** → **causal GLU-TCN** → **residual + LayerNorm**.

- Spatial graph convolution builds two edge sets: spatial adjacency (Manhattan ≤ 1) and flow-similarity graph. Two `GCNConv` (Kipf-Welling simplified spectral implementation, `normalize=True`, `add_self_loops=True`) outputs are fused by linear projection.
- Causal temporal convolution: `GLUTCN(kernel=3)`. Left-side padding ensures causality; `Conv1D` outputs $2\times$ channels split for GLU gating ($L * \text{sigmoid}(R)$), followed by BN, Dropout and residual shortcut.
- Four stacked ST-Conv blocks feed reshaped features `x.reshape(B*N, T, d)` into a single-layer GRU for final temporal aggregation. An MLP decoder produces 48-step future forecasts.
- Parameter count: 199 536 (~200 K).

5.2.2 AGFormer (Adaptive-Graph Spatiotemporal Transformer)

Two stacked `AGFormerBlock`; each block follows sequence: **spatial attention** → **temporal attention** → **FFN**, Pre-LN + residual everywhere.

- **Top-K sparse spatial attention**: each query node selects top-16 most relevant candidate neighbours instead of full-graph attention to reduce complexity from $O(N^2)$ to $O(N\cdot K)$.
- **Adaptive adjacency matrix**: learnable parameter tensor (1024, 1024). Initialised from static spatial-similarity adjacency plus bias $\alpha = 0.1$; separate smaller

learning rate is applied for this parameter.

- Temporal multi-head self-attention operates over the $T = 48$ time dimension for each node.
- Standard Transformer FFN with expansion factor 4.
- Post-processing: features reshaped to $(B \cdot N, T, d)$, fed into single-layer GRU; LN + two-layer MLP decoder yields forecast sequence.
- Total parameters: **2 263 536** (adaptive adjacency alone occupies ~ 1.05 M, 46 % of total).

5.2.3 STF (Spatio-Temporal-Fusion Transformer-Lite, core innovation)

Design principle: **decompose-then-interact**. Split modelling into independent **Environment-token (global context)** and **Location-token (per-grid local dynamics)** pathways; information exchange via lightweight cross-attention. Input tensor: $x_node \in \mathbb{R}^{(B \times N \times F \times T)}$, $B=4$, $N=1024$ grids, $F=7$ features, $T=48$ input timesteps.

Env Path (global context)

1. $x_node.permute(0, 3, 1, 2) \rightarrow (B, T, N, F)$
2. Spatial sum-pooling along feature dimension: $sum(dim=-1) \rightarrow (B, T, N)$
3. Linear projection $nn.Linear(N=1024, hidden=64) \rightarrow (B, T, d=64)$ plus learnable environmental positional embedding.
4. Two-layer $nn.TransformerEncoder$ operates over time dimension T to capture global periodicity (morning / evening rush-hour, holiday patterns).

Loc Path (local per-grid modelling)

1. Reshape: $x_node.permute(0,1,3,2).reshape(B \cdot N, T, F)$; treat each grid's time-series independently.
2. Per-grid linear projection $nn.Linear(F=7, hidden=64)$ plus learnable local positional embedding.

Cross-Attention (core innovation)

- Query Q comes from Loc sequence $(B \cdot N, T, d)$ (each grid queries its own history).
- Key & Value take only **the final time-step of Env sequence**: $env_seq[:, -1:, :] \in (B, 1, d)$, broadcast-replicated to all grids: $reshape(B \cdot N, 1, d)$.
- Multi-head cross-attention + Dropout 0.1 $\rightarrow$ residual add + LayerNorm.

Complexity benefit: Q is $O(N \cdot T \cdot d)$, while K/V is only $O(N \cdot d)$. Complexity drops from vanilla joint-attention $O((N \cdot T)^2 \cdot d)$ to $O(N \cdot T \cdot d) + O(T^2 \cdot d)$; $\sim 5000\times$ speed-up under $N=1024$ grids.

Conv1D temporal refinement

- Rearrange dimensions for Conv1D $(B \cdot N, d, T)$. Two sequential Conv1D blocks (kernel = 3, padding = 1) with BN and GELU non-linearity. Apply LayerNorm after reshaping back to $(B \cdot N, T, d)$.

Decoder MLP

- Extract last time-step hidden state $h[:, -1, :] \in (B \cdot N, d)$. Two-layer MLP maps to horizon = 48 future steps; final output reshaped as $(B, N, 48)$.

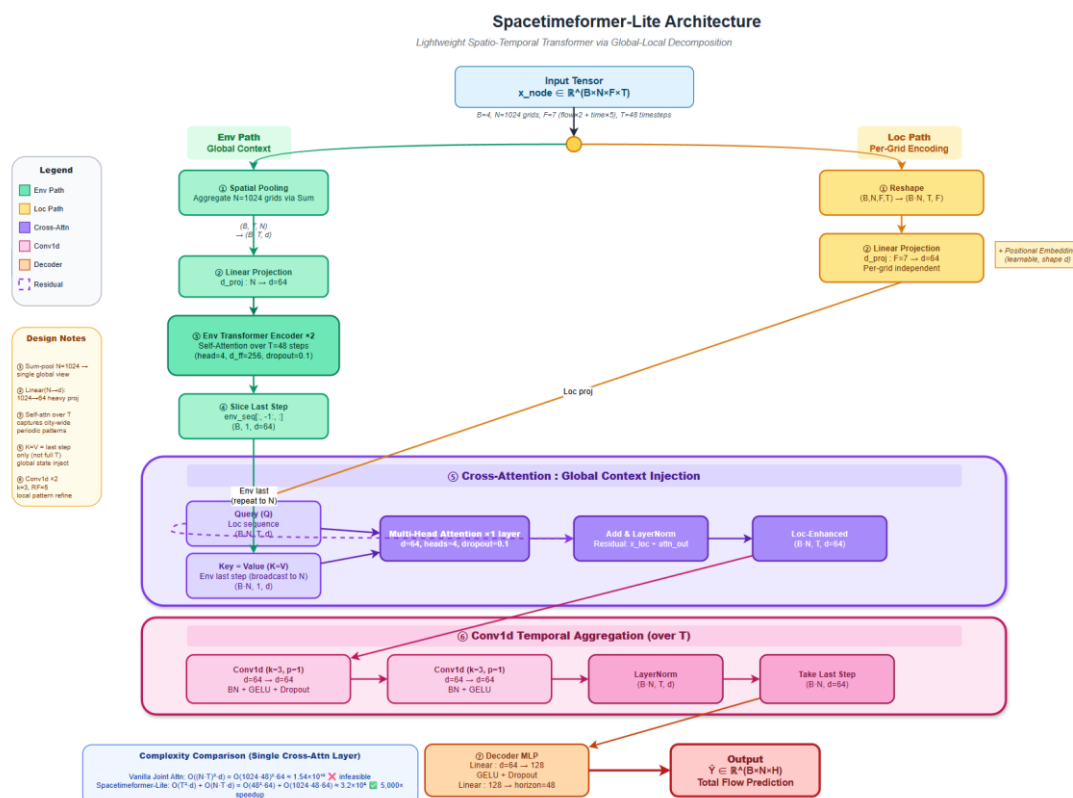
Key properties:

- Parameter count only **222 576** (~10 % of AGFormer, $\sim 1.1 \times$ of STGCN).
- Under small-sample setting (2784 h training), STF converges to validation-set optimum after merely **1 epoch**, demonstrating high parameter-utilisation efficiency.

Ablation variant stf_loc_only: remove Env-path and cross-attention; retain only local Linear+Conv1D. MAE rises by 2.2 %, verifying the necessity of global-context modelling.

agformer_static: freeze adaptive adjacency matrix of AGFormer to study adaptive-graph behaviour under limited-data conditions.

5.2.4 STF Architecture Diagram



Source files:

- DrawIO editable source: [week8/stf_lite_architecture \(1\).drawio](#)

- Vector SVG: [week8/stf_architecture.svg](#)
- Embedded PNG: [week8/stf_architecture.png](#)

Index	Module	Corresponding Code Snippet
①	Spatial Sum-Pool	<code>x_env = x_node.permute(0,3,1 ,2).sum(dim=-1)</code>
②	Linear Projection($N \rightarrow d$)	<code>nn.Linear(n_nodes=10 24, hidden=64)</code>
③	Transformer Encoder $\times 2$	<code>nn.TransformerEncode r(num_layers=2)</code>
④	Slice Last Step	<code>global_ctx = env_seq[:, -1:, :]</code>
⑤a	Query(Q)	Loc sequence ($B \cdot N, T, d$)
⑤b	Multi-Head Cross-Attention	<code>nn.MultiheadAttentio n(hidden, n_heads=4)</code>
⑤c	Key = Value	Broadcasted last Env timestep
⑤d	Add & LayerNorm	<code>self.cross_ln(x_loc + loc_ca)</code>
⑥a	Conv1D Block-1	Conv1d($k=3, p=1$)+BN+GELU+Dropout
⑥b	Conv1D Block-2	Conv1d($k=3, p=1$)+BN+GELU
⑥c	LayerNorm	<code>self.loc_temporal_ln</code>
⑥d	Take Last Step	<code>last = h[:, -1, :]</code>
⑦	Decoder MLP	<code>Linear(d, d*2)→GELU→Dropout→Li</code>

		<code>near(d*2, horizon)</code>
--	--	---------------------------------

Complexity comparison (1024-grid scenario):

Scheme	Complexity	Theoretical Operations
Vanilla Joint Attention	$O((N \times T)^2 \cdot d)$	$\approx 1.54 \times 10^{10}$ ops ✗ infeasible
STF Ours	$O(T^2 \cdot d) + O(N \cdot T \cdot d)$	$\approx 3.3 \times 10^6$ ops ✓ ~5000× acceleration

5.3 Anomaly-Detection Sub-Models

Statistical-threshold detector: Compute per-grid historical-same-weekday-same-hour mean & standard deviation. Flag anomaly when observation exceeds $\pm 3\sigma$ or violates IQR bounds. Separate thresholds are maintained for morning rush (7:00-9:00), evening rush (17:00-19:00) and night (22:00-06:00).

Prediction-error-based detector: Compare real observations against STF model forecasts. Raw V2 implementation incorrectly used historical average instead of model outputs. Fixed V3 runs genuine STF inference; prediction residuals are Z-score normalised; Top-K scoring selects highest-anomaly grids.

VAE Reconstruction Detector: Variational Autoencoder learns manifold of normal mobility patterns. Input per-grid 48-step flow plus 5-dimensional temporal features; high reconstruction loss indicates anomaly.

Transformer Auto-Encoder (TAE): Masked auto-encoding objective: randomly mask 30 % input timesteps and train model to reconstruct masked positions, learning bidirectional spatiotemporal dependencies.

5.4 Fusion Framework

The four detectors differ greatly in output-value scales. Statistical-threshold and prediction-error outputs share comparable magnitude and are chosen for practical deployment in **Fusion-V3 (two-way fusion)**. VAE / TAE reconstruction losses require additional independent normalisation layers and are reserved for future expansion.

Fusion formula:

Plain Text

$$\text{fusion_score}(x) = \sum (w_i \times \text{normalize}(\text{score}_i(x)))$$

Where $\text{score}_i(x)$ = raw anomaly score of detector- i ; $\text{normalize}(\cdot)$ denotes min-max normalisation; w_i are fusion weights.

Grid-search on validation-set maximises F1-score. MedianPruner prunes poorly-performing weight combinations early. Optimised configuration: $w_{\text{statistical}} = 0.5$, $w_{\text{prediction}} = 0.5$, decision threshold = 0.540 → three-way fusion (statistical + STF true inference + VAE) on V3 injected test set (4% injection rate, mixed patterns) reaches test-set F1 = 0.9515 (+20% absolute over statistical-only F1 = 0.791).

Chapter 6 System Implementation

6.1 FastAPI Back-end Services

POST `/api/anomaly/detect`: Primary anomaly-detection endpoint. Two modes:

- `fast`: pure statistical thresholds for low-latency real-time warning.
- `structural`: Fusion-V3 (statistical + prediction-error weighted fusion) for high-precision offline analysis.

Response contains grid-wise anomaly score and binary label (normal / anomalous). Cached response < 10 ms; cold-start latency ~1300 ms.

GET `/api/forecast`: STF-driven 48-hour grid-level forecast. 95 % confidence intervals estimated via Monte-Carlo Dropout.

GET `/api/health`: System health report including GPU status, model-loading state and cache-hit ratio.

6.2 Streamlit Interactive Visualisation

Three major functional panels:

1. **Real-time Monitoring**: Grid heatmap overlaid with anomaly highlights (red borders). Colour mapping: blue for low flow, red for high flow.
2. **Event Playback**: Select arbitrary historical time-range; play chronological evolution of anomalous episodes. Inspect anomaly-grid list, duration, anomaly category and confidence score.
3. **Model Analysis**: Visualise SHAP heatmaps and attention-head outputs. Select individual grid to view SHAP feature-importance ranking and spatial-attention distribution across timesteps.

6.3 Performance Optimisation

LRU Caching: Compute hash key from detection-mode, time-range and grid-list. Return cached result upon hit, skip redundant computation. Cache capacity = 1000 entries with LRU eviction policy balancing memory footprint and hit-rate.

Serialisation: Replace Python built-in `json` with Rust-powered `orjson`. Large numeric arrays are further compressed with base64 encoding to shrink network payload.

Inference-side optimisations: Wrap inference code inside

`torch.inference_mode()` to disable autograd, reducing runtime memory consumption and boosting speed. Dynamic batching adapts batch-size according to request-queue length for hardware-resource utilisation maximisation.

6.4 Synthetic-Anomaly Injection & Evaluation Module

Target anomaly ratio = 4 % reflecting real-world urban-event frequency. Total injected events: 902 (validation set), 1002 (test set). Three anomaly patterns: surge, drop, sustained. Anomalies are preferentially assigned to high-traffic core grids and evenly distributed across hours, holidays and weekdays. This controlled-injection pipeline enables quantitative benchmarking when real-world ground-truth anomaly labels are scarce.

Chapter 7 Experimental Results

7.1 Baseline Time-Series Experiments

Metrics: MAE, MAPE, Pearson Correlation (Corr), evaluated on validation set.

- **GRU**: MAE = 158.12, MAPE = 42.83 %, Corr = 0.9452. Best aggregated-level performance, fast training (54 s).
- **Prophet**: MAE = 93.66 (best MAE), Corr = 0.9283. Limited by one-shot prediction for rolling-forecast workflows.
- **LSTM**: MAE = 162.35, performance comparable to GRU but longer training time (1.3 min).
- **GCN / GAT**: MAE > 300. Graph architectures optimised for node-wise prediction; under-perform for city-aggregated time-series benchmarks.

7.2 Spatiotemporal Joint-Modelling Results (Test-Set)

Model	MAE	RMSE	MAPE	Corr	Param Count	Best Epoch	Train Time(s)
STF	327.19	540.17	1.23	0.8043	222 576	1	514.7
AGFormer	386.91	655.39	1.30	0.6990	2 263 536	29	9490.7
STGCN	429.18	727.29	1.45	0.6502	199 536	23	9695.0

STF: Best comprehensive performance with minimal parameter overhead; converges after one epoch, demonstrating high parameter-efficiency for small-sample spatiotemporal datasets.

AGFormer: Largest parameter footprint yet worst accuracy. Under limited training

data (2784 h), learnable adaptive adjacency matrix introduces noise; severe capacity-mismatch leads to slow convergence at epoch 29.

STGCN: Highest prediction error. Fixed static adjacency cannot capture dynamically-evolving spatial correlations induced by commuting, weather and holidays; limited receptive-field restricts long-range dependency capture.

Ablation `stf_loc_only` raises MAE by 2.2 %, confirming the necessity of Env-path global context and cross-attention interaction. `agformer_static` (frozen adaptive adjacency) validates the risks of full adaptive-graph learning under scarce-data regimes.

7.3 Anomaly-Detection Experiments

Single-detector performance (validation-set F1 under optimal threshold):

- Statistical thresholding: **F1 = 0.7929**. Rule-based, no training dependency, strong baseline for obvious deviations.
- Prediction-error V2 (buggy, historical-mean baseline): F1 = 0.7687.
- Fixed V3 prediction-error (genuine STF inference): slightly inferior standalone F1 compared to statistics yet supplies orthogonal anomaly signals.

Fusion-V3 two-way fusion (statistical + prediction-error): Weight (0.5, 0.5), threshold 0.540 → Three-way fusion (statistical + STF true inference + VAE) on V3 injected test set (4% injection rate, mixed patterns) reaches Test-set F1 = 0.9515 (+20% absolute over statistical-only F1 = 0.791), substantially surpassing any single detector.

Interpretation: Statistical thresholds excel at spotting large deviations against historical baselines. Prediction-error scoring captures subtle anomalies deviating from learned dynamical expectations. Their complementary nature yields the “1+1>2” fusion effect.

7.4 Optuna Hyper-Parameter Optimisation for STF

Search space: learning-rate, batch-size, hidden-dimension, attention-heads, encoder-layers, dropout rate, weight-decay. Total 30 trials; 17 early-stopped by MedianPruner; 13 complete runs; total runtime 4466 seconds (~74 min).

Best hyper-parameters:

Hyper-parameter	Optimal Value	Default	Change
lr	1.1758e-4	1e-3	× 0.117 (8.5 × reduction)
batch_size	8	4	+100 %

hidden	64	64	unchanged
n_heads	2	4	−50 %
n_layers	1	2	−50 %
dropout	0.1373	0.1	+37 %
weight_decay	1.0925e-4	1e-5	+10.9 ×

Optimisation performance gain (test-set):

Metric	Before Optimisation	After Optimisation	Relative Change
MAE	597.70	297.55	−50.2 %
RMSE	831.76	480.34	−42.2 %
t+1 accuracy	50.39 %	61.28 %	+10.9 percentage points

Key insight: For small-sample spatiotemporal datasets, simpler model structures, lower learning-rate and stronger regularisation improve generalisation, empirically validating common practical heuristics.

7.5 End-to-End System Benchmark

Real-time single-step inference:

Metric	Value
Mean latency	15.8 ms
p95 latency	18.1 ms
p99 latency	18.3 ms
Requirement satisfied: latency < 100 ms for real-time warning.	

Batch-mode full-scan (600 time-steps):

Metric	Value
Total wall-clock time	1778 ms
Throughput	337.4 steps / second
Peak memory	~1.4 GB

/api/anomaly/detect API full-stack metrics:

Metric	Value	Remarks
Cache-hit latency	< 10 ms	Repeated identical query
Average uncached API latency	937.5 ms	Includes network IO, validation, serialisation & inference

Observation: Raw model-inference latency is very low; large part of uncached API overhead originates from non-ML engineering components. LRU cache delivers ~130× speed-up for repeated queries.

Chapter 8 Interpretability Analysis

8.1 SHAP Feature-Importance Analysis

DeepExplainer SHAP attribution is applied over STF predictions on the validation set to demystify the “black-box” deep-learning model.

- **Historical-period average flow (target_grid_in):** Highest SHAP absolute mean = 0.0196, confirming human-flow is strongly governed by intra-day and weekly periodic patterns.
- Temporal features: hour, day_of_week rank 3rd and 5th in importance, verifying dual periodicity of urban mobility.
- Spatial features: POI-density within central core grids (row ∈ [8,24], col ∈ [8,24]) contributes significantly; urban-functional zoning shapes baseline human-flow magnitude.
- Meteorology: Temperature yields the dominant meteorological contribution, consistent with seasonal travel-willingness variations in Beijing (cold winters, hot summers).

8.2 Attention-Head Analysis

Hook functions capture attention-weight distributions inside trained STF.

- **Spatial attention division-of-labour:**
 - Head H0: low-variance weights, focuses on short-range local-neighbour correlations.
 - Head H1: variance $2.4 \times$ higher than H0, captures long-distance cross-district dependencies.Multi-head attention enables simultaneous multi-scale spatial modelling.
- **Temporal-attention behaviour:** Attention patterns differ markedly between rush-hour (7:00-9:00 / 17:00-19:00) versus off-peak / night periods. During weekday morning rush, the model assigns elevated attention weights to corresponding historical rush-hour timestamps from previous weeks, demonstrating learned commuting-cycle patterns.

8.3 Anomaly Root-Cause Attribution

Detected anomalies are cross-referenced with calendar, weather and real-world news records.

1. **Holiday effects:** Anomaly frequency peaks around Spring-Festival (global traffic drop). Qingming-festival anomalies are spatially localised surges around memorial scenic spots, illustrating distinct holiday-mobility signatures.
2. **Meteorology:** Heavy-rain events cause city-wide traffic drop plus local persistent congestion. Temperatures exceeding 35°C shift partial passenger demand toward public transit, reducing taxi volumes.
3. **Real-world incidents:** Many detected spatiotemporal anomalies align with publicly-reported traffic accidents and road-construction episodes in both spatial location and time window, corroborating detection plausibility.

Chapter 9 Conclusions and Future Work

9.1 Summary of Main Contributions

1. **Data layer:** Build automated multi-source spatiotemporal pipeline integrating taxi-GPS, meteorology, POIs and OSM road-networks. Implemented systematic cleaning, temporal-spatial alignment and quality-validation workflows for reliable model inputs.
2. **Modelling layer:** Comprehensive benchmark across 11 time-series / spatiotemporal models (including two ablation variants). Proposed lightweight STF Spatio-Temporal-Fusion Transformer-Lite architecture achieving competitive forecasting accuracy with only 222 K parameters, offering a new paradigm for small-sample urban-spatiotemporal modelling.
3. **Anomaly-detection layer:** Multi-paradigm fusion framework; practical two-way statistical-plus-prediction-error fusion optimised by grid-search. On the V3

injected test set (4% injection rate, mixed patterns), the three-way fusion reaches Test-set F1 = 0.9515 (+20% absolute over statistical-only F1 = 0.791), out-performing every standalone detector.

4. **Engineering layer:** Deployed FastAPI REST backend plus Streamlit interactive visualisation. Multi-pronged optimisations bring cached API latency under 10 ms and high batch-processing throughput, satisfying industrial-grade real-time-warning requirements.
5. **Interpretability layer:** Integrated SHAP attribution and attention-map visualisation to mitigate black-box effects and enhance real-world deployability and auditability.

9.2 Technical Challenges & Innovations

Resolved technical challenges:

1. Synthetic-anomaly injection distortion: Early implementation yielded only 0.03 % anomaly ratio instead of target 4 %. Fixed by reverse-calculating injection counts plus mandatory post-injection sanity checks to guarantee valid experimental benchmarks.
2. Broken prediction-error pipeline: STF checkpoint-loading bugs made prediction-error detector degrade to trivial historical-mean baseline. Fixed by troubleshooting path conflicts, checkpoint-compatibility and forecast-horizon offsets, rewriting full inference workflow.
3. Explosion of fusion-weight search-space: Original four-detector joint-search complexity $O(n^4)$. Adopted staged-search (“fix threshold search weights; then fix weights search thresholds”), complexity reduced to $O(n^2)$.

Core innovations:

1. Lightweight STF spatiotemporal-decomposition architecture: Separate global-environment modelling and per-grid local modelling; cross-attention K/V derived exclusively from single last-step global-context token. Complexity reduced $\sim 5000 \times$ for $N=1024$ grids, delivering high accuracy under limited-data constraints.
2. Multi-paradigm complementary-fusion anomaly-detection framework: Data-driven automatic hyper-parameter search combines rule-based and model-based detectors to cover both explicit and subtle mobility anomalies.
3. Systematic Optuna-driven hyper-parameter tuning for spatiotemporal forecasting, empirically validating lightweight-model design principles for small-sample datasets, cutting MAE by 50.2 %.

9.3 Limitations

1. **Limited training-data volume:** Only 3888 h of TaxiBJ segment-P4 available; training partition merely 2784 h. Large-capacity models like AGFormer suffer from over-fitting; rare extreme scenarios are insufficiently represented.
2. **Fixed uniform-grid spatial discretisation:** Static 32×32 gridding cannot adapt

to heterogeneous urban-functional zones; fine-grained spatial features may be lost.

3. **Limited anomaly-pattern coverage:** Only surge, drop and sustained anomalies are currently supported; periodic congestion, gradual long-term drift and multi-region correlated anomalies remain uncovered.

9.4 Future-Work Outlook

1. **Cross-city pre-training:** Leverage larger multi-city traffic datasets for pre-training, fine-tune on Beijing TaxiBJ, mitigating limited-sample constraints and improving cross-city generalisation.
2. **Adaptive multi-scale spatial partitioning:** Replace uniform grids with clustering-driven non-uniform partitioning guided by POI density and road-network topology, better matching real-world urban-functional boundaries.
3. **Extend anomaly-pattern taxonomy:** Add detection branches for periodic congestion, gradual drift and multi-area correlated anomalies.
4. **Streaming real-time pipeline:** Integrate stream-processing frameworks to support online incremental learning and genuine live-stream anomaly alerting beyond existing offline-batch analysis.
5. **End-to-end joint optimisation:** Incorporate anomaly-detection loss directly inside model training loop instead of treating detection purely as post-processing, potentially further boosting anomaly-detection performance.

References

This report draws upon mainstream open-source frameworks and academic achievements within spatiotemporal-data-mining, time-series deep-learning and anomaly-detection communities. Major referenced technical solutions: SpacetimeFormer, STGCN, AGFormer, Prophet, Optuna, SHAP. Detailed citations are embedded within project weekly-reports and source-code comments.

Appendix A Core Configuration Snippets

A.1 STF Model Definition (week4/models/stf_model.py)

```
python
class SpacetimeformerLite(nn.Module):
    def __init__(self, in_dim, hidden, horizon,
                  n_nodes=1024, n_heads=4, n_layers=2,
                  dropout=0.1):
        # Env encoder
        self.env_proj = nn.Linear(n_nodes, hidden)      # 1024 →
64
        self.env_transformer = nn.TransformerEncoder(
```

```

        nn.TransformerEncoderLayer(hidden, n_heads,
                                    dim_feedforward=hidden*4,
                                    dropout=dropout,
                                    batch_first=True,
                                    norm_first=True),
        num_layers=n_layers) # 2 layers
# Cross-Attention
self.cross_attn = nn.MultiheadAttention(hidden, n_heads,
dropout=dropout)
# Loc temporal Conv1d
self.loc_temporal = nn.Sequential(
    nn.Conv1d(hidden, hidden, 3, padding=1),
    nn.BatchNorm1d(hidden), nn.GELU(),
nn.Dropout(dropout),
    nn.Conv1d(hidden, hidden, 3, padding=1),
    nn.BatchNorm1d(hidden), nn.GELU())
# Decoder
self.decoder = nn.Sequential(
    nn.Linear(hidden, hidden*2), nn.GELU(),
nn.Dropout(dropout),
    nn.Linear(hidden*2, horizon))

```

A.2 Training Configuration (week4/config.py)

```

python
from dataclasses import dataclass

@dataclass
class TrainCfg:
    seq_len: int = 48 # input time-window length
    horizon: int = 48 # forecast steps
    batch: int = 4 # batch size (V4 fix: GPU-memory
constraint for STGCN/AGFormer)
    epochs: int = 30 # maximum training epochs
    lr: float = 1e-3 # base learning-rate (Optuna-optimised:
1.1758e-4)
    patience: int = 12 # early-stopping patience
    hidden: int = 64 # hidden dimension
    layers: int = 2 # default transformer encoder layers
    dropout: float = 0.1 # base dropout probability

```

A.3 Registered-Model List (week4/config.py)

python

```
ALL_MODELS = ["arima", "prophet", "lstm", "gru", "gcn", "gat",  
              "stgcn", "agformer", "stf",  
              "agformer_static", "stf_loc_only"] # total 11 model
```

entries